

Formal Analysis of Reaction Systems Using Maude Strategies

Demis Ballis¹[0000–0002–1048–1739], Linda Brodo²[0000–0002–4455–2419], Moreno Falaschi³[0000–0002–6659–3828], and Carlos Olarte⁴[0000–0002–7264–7773]

¹ DMIF, University of Udine, Italy

² Dipartimento di Scienze economiche e aziendali, Università di Sassari, Italy
`brodo@uniss.it`.

³ Dipartimento di Ingegneria dell’Informazione e Scienze Matematiche
Università di Siena, Italy `moreno.falaschi@unisi.it`.

⁴ LIPN, CNRS UMR 7030, Université Sorbonne Paris Nord, France
`olarte@lipn.univ-paris13.fr`

Abstract. Reaction Systems (RSs) are computational models inspired by biological systems, in which entities are produced by reactions and may enable or inhibit other reactions. The interaction of an RS with its surrounding environment is modeled by sets of entities, called contexts. This paper introduces an executable semantics for Reaction Systems, formalized as a fully nondeterministic rewrite theory. By generating contexts from all possible subsets of external environmental stimuli at each step, the model explores all potential environmental interactions in parallel, without predetermining any specific context sequence. To mitigate the state explosion caused by this unrestricted nondeterminism, our execution model uses rewriting strategies implemented in the Maude rewriting engine. These strategies allow us to prune the computation tree and discover relevant execution paths, with the aim of synthesizing medical treatments. We show that our model is versatile and supports a wide range of verification tasks, including reachability analysis via strategic rewriting, LTL/CTL model checking, context-sequence synthesis for dynamically deriving environmental inputs, and conditional context generation via perfect-recall strategies. We apply our formal methodology to a breast cancer case study. Our analyses not only confirm existing results in the literature, but also support the validation of new hypotheses.

Keywords: Reaction Systems · Rewriting Logic · Maude · Formal Verification · Model Checking.

1 Introduction

Reaction Systems (RSs) [5] are a bio-inspired computational framework applicable to both biological modeling [2, 3, 7] and theoretical computing foundations [9, 10]. A RS is built from a finite set of entities and a finite set of reactions acting on entities. A reaction is a triple (R, I, P) where R is the set of *reactants* (entities whose presence is needed to enable the reaction), I is the set of

inhibitors (entities whose absence is needed to enable the reaction) and P is the set of *products* (entities that are produced if the reaction is enabled). The behavior of a RS is typically specified as a rewrite system that models *processes interacting* with an external *context* that supplies environment entities at each step. The current system state is determined by the union of the entities coming from the environment with those produced in the previous state. A state transition is then determined by applying all and only the enabled reactions to the entities in the current state.

Rewriting Logic (RL) [13] is a comprehensive semantic framework for modeling complex, nondeterministic systems, efficiently implemented in the high-performance language Maude [6]. The specification unit in RL is a *rewrite theory*, that combines a set of rewrite rules modeling the concurrent transitions of a system, and an equational theory, representing system states as terms of an algebraic datatype. Maude’s default engine executes any applicable rule at each step, potentially causing state explosion in highly nondeterministic systems. To mitigate this problem, Maude is equipped with a strategy language that allows users to define custom execution paths, effectively controlling the rewriting process and restricting the search space instead of freely exploring all possible rule interleavings.

Contributions. This paper presents a formal, executable semantics for RSs, that is encoded as a Maude rewrite theory. The rewrite theory is fully nondeterministic: at each computation step, the context is generated as any possible subset of a fixed pool of external environmental stimuli, allowing the system to explore all potential interactions with the environment in parallel without committing to any specific context sequence in advance. To tame this high degree of nondeterminism and improve performance, the theory is endowed with a strategy language that acts as a filtering mechanism, pruning the computation tree and retaining only those execution paths that the user considers relevant. This modelling framework is particularly versatile, since it supports a wide range of analysis tasks beyond mere simulation. Strategic rewriting enables reachability analysis by systematically exploring all execution paths that meet a given set of constraints, while integration with the Unified Maude Model-Checking tool [15] supports formal verification of temporal properties expressed as LTL or CTL formulas over strategy-constrained fragments of the computation tree. Furthermore, the nondeterministic nature of the context generation mechanism naturally supports context sequence synthesis: rather than fixing the environmental inputs in advance, the framework can be queried to find all context sequences that drive the reaction system to states satisfying a given property.

To demonstrate the applicability of our approach, we evaluate it on the biological case study of [11], which analyses protein signalling networks in HER2-positive breast cancer subtypes under different combinations of monoclonal antibody drugs. Our analyses not only corroborate existing results in the literature but also enable the validation of new hypotheses.

Organization. Section 2 recalls the basics of RSs. Section 3 introduces some basic notions on Rewriting Logic and Maude, while Section 4 presents the Maude executable model for RSs along with the strategy operators used to control nondeterminism within the system. Section 5 evaluates our approach on the biological case study of [11], showing how strategic rewriting, context sequence synthesis, perfect recall strategies (i.e., strategies that conditionally generate new contexts based on the history of results computed so far), and LTL/CTL model checking can be effectively used to reproduce and extend the experimental results of [11]. Finally, Section 6 reviews related work and concludes.

2 Reaction Systems

In this section we recall some basic notions about RSs that are relevant to this work. For a full discussion refer to [5].

We use the term *entities* to denote generic molecular substances (e.g., atoms, ions, molecules) that may be involved in biochemical reactions. Let S be a finite set of entities, a *reaction* in S is a triple (R, I, P) , where R, I , and P are finite sets of entities such that $R, I, P \subseteq S$ and $R \cap I = \emptyset$. The sets R, I, P respectively denote the sets of *reactants*, *inhibitors*, and *products* of the reaction. Due to biological considerations, the sets R and P are usually nonempty: there is no creation from nothing ($R \neq \emptyset$), and if a reaction takes place then something is produced ($P \neq \emptyset$). By $\text{rac}(S)$ we denote the set of all reactions over S .

A reaction can take place whenever all of its reactants are present in the current state while none of its inhibitors is present. If this happens, the reaction is *enabled* and creates its products which will be available in the next state. Formally, given a set $W \subseteq S$, and a reaction $a = (R, I, P) \in \text{rac}(S)$, the *result* of a on W is defined by

$$\text{res}_a(W) \triangleq \begin{cases} P & \text{if } \text{en}_a(W) \\ \emptyset & \text{otherwise} \end{cases}$$

where the *enabling* predicate is defined by $\text{en}_a(W) \triangleq (R \subseteq W) \wedge (I \cap W = \emptyset)$. Let $A \subseteq \text{rac}(S)$ be a finite set of reactions. The result of A on W , denoted by $\text{res}_A(W)$, is defined by: $\text{res}_A(W) \triangleq \bigcup_{a \in A} \text{res}_a(W)$. A reaction system (RS, in short) is a pair $\mathcal{A} = (S, A)$ such that S is a finite set of entities and $A \subseteq \text{rac}(S)$.

RSs are based on three assumptions: *no permanency*, i.e., any entity vanishes unless it is sustained by a reaction; *no counting*, the basic model of RSs is very abstract and qualitative, i.e. the quantity of entities that are present in a cell is not considered; *threshold nature of resources*, it is assumed that either an entity is available for all reactions, or it is not available at all. This implies that nonempty intersection of sets of reactants of enabled reactions never generates conflicts, which is a major difference with other models for concurrency (e.g., Petri nets). Mimicking living cells, which constantly interact with the environment, a RS is inherently an open system whose behavior is formalized through the notion of *interactive process*, i.e., a process that may react to external stimuli. In our

setting, an *environment* is a finite set of entities $Env \subseteq S$. Entities in Env denote external stimuli that can feed the RS. An interactive process is then formally defined as follows.

Definition 1 (Interactive Process). Let $\mathcal{A} = (S, A)$ be a RS and let $n \geq 0$. Let $Env \subseteq S$ be an environment. An n -step interactive process in $\mathcal{A}$ w.r.t. Env is a pair $\pi = (\gamma, \delta)$ s.t. $\gamma = \{C_i\}_{i \in [0, n-1]}$ is the context sequence and $\delta = \{D_i\}_{i \in [0, n]}$ is the result sequence, where $C_i \subseteq Env$, $D_i \subseteq S$ for any $i \in [0, n]$, $D_0 = \emptyset$, and $D_i = res_A(D_{i-1} \cup C_{i-1})$ for any $i \in [1, n]$.

The context sequence γ represents the environment interacting with the RS. It specifies the *input* that comes from the environment. The result sequence δ is entirely determined by γ and the reactions in A . As expected, distinct context sequences may lead to distinct outcomes for the same reaction set A , but a context sequence γ determines a *unique* result sequence δ .

Example 1. Let $\mathcal{A} = (S, A)$ be a RS such that $S = \{u, v, w, s_1, s_2\}$, $Env = \{s_1, s_2\}$ and $A = \{a_1, a_2, a_3, a_4\}$ where

$$\begin{aligned} a_1 &= (\{s_1\}, \{s_2\}, \{u, v\}) & a_2 &= (\{s_1, s_2, u, v\}, \emptyset, \{w\}). \\ a_3 &= (\{w\}, \{s_1, s_2\}, \{u\}) & a_4 &= (\{u, v, s_1\}, \{s_2\}, \emptyset, \{v\}). \end{aligned}$$

Consider a 3-step interactive process $\pi = (\gamma, \delta)$ where $\gamma = C_0, C_1, C_2$ with $C_0 = \{s_1\}$, $C_1 = \{s_1, s_2\}$ and $C_2 = \{s_2\}$. Then $\delta = D_0, D_1, D_2, D_3 = \emptyset, \{u, v\}, \{w\}, \emptyset$.

3 Rewriting Logic and Maude

Maude [6] is a formal language based on rewriting logic (RWL) [13] well suited to model software systems. In RWL, an equational theory defines system states (as algebraic terms), while rewrite rules define system behavior (as local state transitions).

An *equational theory* $\mathcal{E}$ in Maude is defined by a functional module that specifies data types through operators and equations organized in an order-sorted structure. This structure consists of a set of sorts Γ and a poset $(\Gamma, <)$ that models subsort relations. Operators are declared with the prefix syntax $\text{op } f : s_1 \dots s_n \rightarrow s, n \geq 0$, where $s_1 \dots s_n$ are the sorts of the operator arguments and s is the result sort. Operators with no arguments are called *constants*. Operators can also be declared in mixfix notation using underscores as placeholders (e.g., $_ : s_1 s_2 \rightarrow s$). Binary operators may include algebraic axioms such as associativity, commutativity, and identity; we denote the set of all axioms attached to the operators in the equational theory by Ax . Operators are naturally partitioned in two classes: *constructors*, which are used to define (irreducible) data values, and *defined* operators, which are used to specify expressions that are evaluated away via equational simplification.

We also consider a family of Γ -sorted variables $\mathcal{V}$, where the notation $x : s$ denotes a variable x of sort s . Terms are either variables or applications of

operators to lists of terms. *Patterns* are terms that do not contain defined operators. A *substitution* $\sigma = \{x_1/t_1, x_2/t_2, \dots, x_n/t_n\}$ is a (sort-preserving) mapping from the set of variables $\mathcal{V}$ to the set of all terms, which is equal to the identity everywhere except for the set of variables $\{x_1, \dots, x_n\}$. A substitution $\sigma = \{x_1/t_1, x_2/t_2, \dots, x_n/t_n\}$ is *ground* if $t_1, \dots, t_n$ do not contain variables.

Equations are specified using the syntax `ceq l = r if C`, where l and r are terms whose sorts belong to the same connected component of $(\Gamma, <)$ and C is a conjunction of equations that must be satisfied in order for the equation to be applied. When the condition C is empty, we simply write `eq l = r`. Equations are used to simplify terms into their canonical forms modulo $\mathbf{Ax}$. This is achieved by orienting the equations from left to right to rewrite (modulo $\mathbf{Ax}$) the term to be simplified until no further reductions are possible⁵.

A *rewrite theory* $\mathcal{R}$ extends the equational theory $\mathcal{E}$ with a set of rewrite rules R , which are specified within a Maude system module. Rewrite rules are defined by the syntax `cr1 [lbl] l : => r if C`, where `lbl` is a label that identifies the rule, l and r are terms whose sorts belong to the same connected component of $(\Gamma, <)$, and C is a conjunction of equations and rewrite expressions that must be satisfied to enable a rewrite step. When the condition C is empty, we simply write `rl [lbl] : l => r`. The term l (resp., r) is called *left-hand side* (resp., *right-hand side*) of the rule `cr1 [lbl] l : => r if C` (resp., equation `ceq => r if C`).

Rewrite theories formalize concurrent and nondeterministic software systems that evolve via equational rewriting modulo the equational theory $\mathcal{E}$. Each transition $t \xrightarrow{1}_{R, \mathcal{E}} t'$ involves simplifying the term t to its canonical form before applying the rule identified by the label l . A *computation* in $\mathcal{R}$ corresponds to a (possibly infinite) rewrite sequence $t_0 \xrightarrow{l_0}_{R, \mathcal{E}} t_1 \xrightarrow{l_1}_{R, \mathcal{E}} \dots$, where terms t_i are called *states*. A *computation tree* $T_{\mathcal{R}}(t_0)$ is a tree whose branches represent all computations originating from the initial state t_0 . Throughout this paper rewrite theories are also called Maude specifications.

The Maude strategy language [16] is an execution layer built on top of the Maude rewrite engine. It enables users to control rule application by composing combinators—such as sequencing, conditionals, and iteration—thereby transforming a fully nondeterministic rewrite theory into a user-controlled execution model. A strategy S for a rewrite theory⁶ $\mathcal{R}$ is an expression that is built using the following abstract syntax.

`fail | idle | lbl[ρ] | S1 ; S2 | (S1 | S2) | match P s.t. C | S1 ? S2 : S3 | S1* | S1+ | not(S1)`

where `lbl` is a rewrite rule label, ρ is a ground substitution, S_1 , S_2 , and S_3 are strategies, P is a pattern, and C is an equational condition.

Semantically, a strategy S is a transformation that maps an input state t to the set of all states reachable from t through the computation dictated by S .

⁵ To ensure the existence of a unique canonical form for each term, the equational theory is usually required to be convergent and coherent modulo $\mathbf{Ax}$ [8].

⁶ Here we present the language subset needed for specifying strategies on RSs; however, all Maude strategy operators are fully supported without restriction.

Rules to be executed are given by the strategy operator in S whose meaning is as follows. The **idle** strategy always succeeds and returns the state t unaltered as the only result. In contrast, the **fail** strategy always fails for any given state, always producing an empty set of resulting states. The rule label operator $\text{lbl}[\rho]$ provides a fine control over rewrite rule applications: it attempts to rewrite the term t by applying the rule with label lbl , where the substitution ρ is used to instantiate some (or all) of the rule's variables before the rule application. The sequential operator $S_1; S_2$ models strategy concatenation by first applying S_1 to state t and then applying S_2 to the states yielded by S_1 . The choice operator $S_1 \mid S_2$ executes S_1 or S_2 on the state t , and the results are both those of S_1 and those of S_2 . The **match** $P \text{ s.t. } C$ operator implements testing capabilities: the test succeeds delivering the singleton $\{t\}$, when (i) there is a match (modulo the equational theory) between the pattern P and the whole state t with matching substitution θ and (ii) the instantiated equational condition $C\theta$ holds. Otherwise the test fails and $\emptyset$ is delivered as result. Conditional strategies are implemented via the operator $S_1 ? S_2 : S_3$. This operator executes S_1 on state t and then S_2 on its results, but if S_1 does not produce any, it executes S_3 directly on t . The strategy S_1^* (resp., S_1^+) specifies the transitive and reflexive closure (resp., the transitive closure) of the strategy S_1 . Finally, the operator $\text{not}(S_1)$ is just a convenient notation for the strategy $S_1 ? \text{fail} : \text{idle}$.

To execute a given strategy S on an input term t , Maude is equipped with the **srew** command, whose syntax is **srew** $[n]$ t **using** S , where n is an optional argument that limits the number of solutions returned.

4 A Maude Specification for Reactions Systems

In this section, we outline a Maude specification that serves as an executable semantics for the RS model introduced in Section 2, providing a formal and computable implementation for interactive processes. More precisely, given a RS $\mathcal{A} = (S, A)$, we formulate a rewrite theory $\mathcal{R}_{\mathcal{A}}$ that includes: (i) an equational theory $\mathcal{E}_{\mathcal{A}}$ that implements the basic data structures and set-theoretic operations which are required to model RS states; (ii) a set of rewrite rules $R_{\mathcal{A}}$, which acts on top of $\mathcal{E}_{\mathcal{A}}$, to specify the system behavior of $\mathcal{A}$. We say that $\mathcal{R}_{\mathcal{A}}$ is the *Maude encoding* of $\mathcal{A}$.

The core type structure along with the most relevant operators of $\mathcal{E}_{\mathcal{A}}$ and the rule set $R_{\mathcal{A}}$ are shown in Figure 4.⁷

Entities in $\mathcal{E}_{\mathcal{A}}$ are modeled as constants of sort **Entity**, while entity sets (sort **Entities**) are formed via an associative, commutative juxtaposition operator $_ _$ with identity **empty** (Lines 5–6). Idempotency equation in Line 7 enforces the standard set semantics by collapsing duplicate entities. The sort **Sequence** and its associative operator $_, _$ model an ordered, comma-separated list of entity sets, which will be used to represent both the context sequence supplied by

⁷ Due to space constraints, we omit standard auxiliary definitions (such as the equational definitions for the usual set operators **subset** and **isDisjoint**). The complete Maude source code is available at GITHUBSITE.

```

1  sorts Entity Entities Sequence Reaction Reactions .
2  subsort Entity < Entities < Sequence .
3  subsort Reaction < Reactions .
4
5  op empty : -> Entities .
6  op _ : Entities Entities -> Entities [assoc comm id: empty] .
7  eq x x = x .
8
9  op _ : Sequence Sequence -> Sequence [assoc id:nil] .
10 op nil : -> Sequence .
11 op [_ , _ , _] : Entities Entities Entities -> Reaction .
12
13 op empty : -> Reactions .
14 op _ ; _ : Reactions Reactions -> Reactions [assoc id: empty] .
15
16 op en : Reaction Entities -> Bool .
17 eq en([R,I,P],T) = (R subset T) and-then isDisjoint(I,T) .
18
19 op res : Reaction Entities -> Entities .
20 eq res([R,I,P],T) = if en([R,I,P],T) then P else empty fi .
21
22 op res : Reactions Entities -> Entities .
23 eq res(empty, T) = empty .
24 eq res((r ; Rs), T) = res(r, T) res(Rs, T) .
25
26 op <_ | _> : Reactions Sequence Sequence -> State .
27
28 crl [next] : < A | Ds , D | Cs > => < A | Ds , D , res(A, C D) | Cs , C >
29                                     if C C' := Env

```

Fig. 1. The rewrite theory $\mathcal{R}_A$.

the environment and the result sequence accumulated during computation. The identity `nil` specifies the empty sequence (Lines 9–10). Since the sort `Entity` is a subsort of `Entities` which in turn is a subsort of `Sequence` (Line 2), any entity or entity set is also a term of sort `Sequence`. For instance, the term `egf e, e` is a well-typed sequence provided that `egf` and `e` are terms of sort `Entity`. Reactions are terms of sort `Reaction` of the form `[R,I,P]`, where `R`, `I`, and `P` are sets of entities that respectively specify reactants, inhibitors and products of the reaction (Line 11).

The sort `Reactions` and its associated operator `_ ; _` model an ordered, semicolon-separated list of reactions representing the full reaction set A of a RS. The constant `empty` is again used to specify an empty reaction list (Lines 13–14). Thanks to the subsort relation `Reaction < Reactions` (Line 3), any individual `Reaction` term is also a valid `Reactions` term.

The function `res([R, I, P], T)` (Lines 19–20) computes the output of a single reaction (R, I, P) fired against a given entity set T (i.e., $res_{(R,I,P)}(T)$), delegating the firing condition check to the auxiliary predicate `en([R, I, P], T)` (Lines 16–17). Note that `en([R, I, P], T)` is a direct and straightforward implementation of the enabling predicate introduced in Section 2. Consequently, `res` returns the term P (i.e., the produced entities) if `en([R, I, P], T)` evaluates to true, and `empty` otherwise. The function `res(A, T)` (Lines 22–24), overloaded from the single-reaction

case, computes the set of entities $res_A(T)$ by applying all the reactions in the reaction list A to T .

System states (or simply states) are terms of sort **State** whose form is $\langle A \mid Ds \mid Cs \rangle$, where A is a term specifying the list of reactions of the RS, Ds is the result sequence accumulated so far, and Cs is the context sequence supplied by the environment.

For a fixed set of external stimuli Env , the behavior of a RS $\mathcal{A} = (S, A)$ is completely captured by the single rewrite rule in Lines 28–29 (namely, the **next** rule). Each computation step with **next** atomically combines two phases: context generation and reaction activation. Context generation is achieved by using the rule condition $C \ C' := Env$ that nondeterministically split Env into a context C and a remainder C' , so every possible subset C of Env can be generated during the application of **next**. Reaction activation is then performed by computing the new result set $res(A, C \ D)$, where C is the freshly generated context and D is the most recently accumulated result. Finally, the new result set $res(A, C \ D)$ and context C are appended to their respective sequences, updating the current state. Hence a computation in $\mathcal{R}_A$ of the form:

$$\langle A \mid D_0 \mid nil \rangle \xrightarrow{\text{next}}_{R_A, \mathcal{E}_A} \langle A \mid D_0, D_1 \mid C_0 \rangle \xrightarrow{\text{next}}_{R_A, \mathcal{E}_A} \langle A \mid D_0, D_1, D_2 \mid C_0, C_1 \rangle \xrightarrow{\text{next}}_{R_A, \mathcal{E}_A} \dots$$

where, at each rewrite step, the **next** rule uses the result set D_i and context C_i to compute the next result D_{i+1} , precisely specifies an n -step interactive process w.r.t. a given environment. Furthermore, computational steps are preserved in both directions: each transition in an interactive process of A corresponds to exactly one rewrite step in $\mathcal{R}_A$, and vice-versa, which provides an operational correspondence between the two computational models.

Example 2. Let $\mathcal{A}=(A,S)$ be the RS of Example 1 where $Env = \{s_1, s_2\}$. Let $\mathcal{R}_A$ be the Maude encoding of $\mathcal{A}$. Then, there exists a computation in $\mathcal{R}_A$:

$$\begin{aligned} \langle A \mid \text{empty} \mid nil \rangle &\xrightarrow{\text{init}}_{R_A, \mathcal{E}_A} \langle A \mid \text{empty}, u \ v \mid s_1 \rangle \\ &\xrightarrow{\text{next}}_{R_A, \mathcal{E}_A} \langle A \mid \text{empty}, u \ v, w \mid s_1, s_1 \ s_2 \rangle \\ &\xrightarrow{\text{next}}_{R_A, \mathcal{E}_A} \langle A \mid \text{empty}, u \ v, w, \text{empty} \mid s_1, s_1 \ s_2, s_2 \rangle \end{aligned}$$

where **empty**, $u \ v, w, \text{empty}$ and $s_1, s_1 \ s_2, s_2$ respectively specify the result and context sequences of the 3-step interactive process γ from Example 1.

Example 2 illustrates one specific 3-step interactive process in the RS $\mathcal{A}$ that can be computed by $\mathcal{R}_A$ w.r.t. the environment $Env = \{s_1, s_2\}$. However, due to the nondeterministic nature of $\mathcal{R}_A$, all the n -step interactive processes w.r.t. Env are computed by $\mathcal{R}_A$. Indeed, each application of the **next** rule nondeterministically computes any possible subset of the environment Env as context, therefore generating $2^{|Env|}$ possible branches at each rewrite step, causing a combinatorial explosion that might quickly become intractable. Below, we tackle this problem by formalizing suitable rewriting strategies in the Maude strategy language. These strategies successfully control nondeterminism and guide the search only towards relevant computations.


```

1 sd se(0,Req) := idle .
2 sd se(s(N),Req) := next [ C <- Req ] ; se(N, Req) .
3
4 sd se-sets(N,Req) := se(N, Req) .
5 sd se-sets(N,(Req,S')) := se(N, Req) | se-sets(N,S') .
6
7 sd se-cond (Ctx,Req,Excl,N) := match < R | Ds | Cs >
8                               s.t. suffixCheck(Ds,Req,Excl,N) = true ;
9                               next [ C <- Ctx ] .
10
11 sd s(0,Req,Excl) := idle .
12 sd s(s(N),Req,Excl) := next ; match < R | Ds | Cs , (Req C) >
13                               s.t. isDisjoint(Excl,(Req C) = true ;
14                               s(N, Req, Excl) .
15
16 sd s-all(N) := s(N,empty,empty) .
17 sd s-atleast(N,Req) := s(N,Req,empty) .
18 sd s-without(N,Excl:E) := s(N,empty,Excl) .
19
20 sd isSteady := match < R | Ds , D , D | Cs > .
21 sd se-steady(Req) := isSteady ? idle : (se(1,Req) ; se-steady(Req)) .
22 sd s-steady(Req,Excl) := isSteady ? idle : s(1,Req,Excl) ;
23                               s-steady(Req,Excl) .
24 sd in(Req) := match < R | Ds , (Req D) | Cs > .

```

Fig. 2. Strategy language for RSs

Rewrite Strategies for $\mathcal{R}_A$. The Maude strategy language supports the definition (via the `sd` operator) of named, parameterized strategies that can be recursive and can freely invoke any (auxiliary) equationally defined function. This expressiveness allows us to design the strategy language in Figure 4 specifically tailored to exploring the iterative processes generated by $\mathcal{R}_A$. The language is based on several strategy constructs whose behavior is as follows.

The recursive strategy `se(N,Req)` (Lines 1–2) takes as input a natural number N and a set of entities `Req` and generates an N -step computation in $\mathcal{R}_A$, where—at each rewrite step—the context injected is exactly `Req`, hence producing a totally deterministic N -step interactive process. This is achieved by recursively applying the `next` rule under the substitution `[C <- Req]`, which enforces the required context at each rewrite. `se-sets(N,S)` (Lines 4–5) is a nondeterministic generalization of `se(N,Req)` that uses the choice operator `_|_` to select, in parallel, every set of entities from the sequence `S` as the fixed context, yielding the corresponding N -step computation.

Contexts can also be generated conditionally via `se-cond(Ctx,Req,Excl,N)` (Line 7), which injects context `Ctx` in the next step whenever the last N result sets of the current result sequence include the set of entities `Req` and exclude `Excl`. The condition is checked by the auxiliary function `suffixCheck(Ds,Req,Excl,N)`⁸, which returns true exactly when this holds. Note that this strategy allows the system to evolve based on results computed in the past, making context injection responsive to the history of the computation.

⁸ The full specification of `suffixCheck` can be found at the project Github site [METER LINK o RIFERIMENTO](#).

The recursive strategy $s(N, \text{Req}, \text{Excl})$ (Lines 11–12) nondeterministically explores all possible N -step computations whose context, at each step, includes all entities in Req and excludes all entities in Excl . Unlike the se strategy, entities in the environment Env beyond Req are allowed, as long as they are not in Excl . This is implemented via a `match` test that (i) checks the inclusion of Req by matching, modulo associativity and commutativity, the last result set in the result sequence against the decomposition $(\text{Req } C)$, and (ii) checks the exclusion of Excl by evaluating the set operation `isDisjoint` on Excl and $(\text{Req } C)$. The strategies $se\text{-all}$, $se\text{-atleast}$, and $se\text{-without}$ are all convenient specializations of $s(N, \text{Req}, \text{Excl})$ (Lines 16–18) obtained by instantiating Req and/or Excl to the empty set. In particular, note that $se\text{-all}(N)$ performs an unrestricted N -step exploration that produces all N -step interactive process w.r.t. Env .

The strategy `isSteady` (Line 20) uses a `match` test to check whether the computation has reached a steady state, i.e., whether the last two result sets in the current result sequence are identical, while `in(Req)` (Line 24) checks whether all entities in Req are included in the most recently computed result set.

Finally, the strategies $se\text{-steady}(\text{Req})$ (Line 21) and $s\text{-steady}(\text{Req}, \text{Excl})$ (Line 22) both rely on the conditional operator `_?_:` together with `isSteady` to iterate until a steady state is reached: at each recursive call, `isSteady` is tested and, if it holds, the strategy halts via `idle`; otherwise, one further recursive step is taken.

In the next section we will show how this strategy language can be effectively used to formally analyze a real-size biological example.

5 Analyzing a Real-size Biological Case Study

We consider the case study from [11] concerning protein signaling networks in breast cancer and the associated drug therapies used to inhibit the metabolic pathways responsible for tumor genesis and progression

The *in silico* experimental analysis of [11] focus on therapies where different drug combinations are tested against distinct breast cancer subtypes. More precisely, the authors in [11] consider: (i) three kinds of HER2-positive subtype breast cancer, identified by the cell lines BT474, HCC1954, and SKBR3; (ii) three types of drugs: erlotinib (e), pertuzumab (p), trastuzumab (t), provided alone or in combination; (iii) two types of input stimuli, EGF and HRG, which start the signaling pathways to be analyzed; (iv) two different treatment periods: short- and long-term drug treatments.

The experiments in [11] searches for the presence of key molecules, which are responsible for tumors, in *attractor* states (in short, attractors). Attractor states represent those states toward which

	Short-term									Long-term					
	BT474			HCC1954			SKBR3			BT474		HCC1954		SKBR3	
	AKT	ERK1/2	p70S6K	AKT	ERK1/2	p70S6K	AKT	ERK1/2	p70S6K	RP56	RB	RP56	RB	RP56	RB
x	1	1	1	1	1	1	1	1	1	1	0	1	0	1	0
e	0	1	1	1	0	1	0	0	0	0	0	0	0	0	0
p	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
t	1	1	1	1	1	1	1	1	1	1	0	1	0	1	0
e,p	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0
e,t	0	1	1	1	0	1	0	0	0	0	0	0	0	0	0
p,t	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
e,p,t	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0

Table 1. Short-term and long-term experiments from [11].

the system tends to evolve

over time. Specifically, the short-term analysis searches for AKT, ERK1/2, and p70S6K in attractors, while the long-term analysis searches for RPS6 and RB.

Table 1 summarizes the results of [11] for the six considered systems: one per kind of cancer cell line, short- and long-term versions. In the first column the drug combinations are reported: x stands for no drug treatment. Each row of the table presents the efficacy of a drug combination under the same input stimulus for the three kinds of cancer. The treatment is effective when the molecules of interest (AKT, ERK1/2, p70S6K, for short-term, and RPS6, RB for long-term) are not present in attractor states, i.e. the cell value in the table is equal to 0.

To reproduce the experiments in Table 1 in our verification framework, we reused the six RSs from our previous work [2], which model the reactions for the short-term and long-term analyses of the considered cancer cell lines, respectively. These RSs were directly derived from the Boolean signaling networks of [11] using the translation method proposed in [4]. The entities within these RSs specify every individual component of the system (such as proteins, enzymes, and other chemicals), while the reactions define their dynamic behavior. For instance, the following code snippet shows a fragment of the constant operator BT474, which specifies the reactions used in the short-term experiment for the cancer cell line BT474:

```
op BT474 : -> Reactions .
eq BT474 = [ akt , empty , akt ] ; [ egf , e p , erbb1 ] ; ...
```

The resulting RSs for the six systems listed in Table 1 (from left to right) consist of 39, 39, 34, 47, 50, and 55 reactions, respectively. Their full specifications, along with the data necessary for readers to fully reproduce the experiments, are available at [GITHUBSITE](#). The environment from which contexts are generated includes the input stimuli EGF and HRG and the three drugs e, p, and t). This environment is modeled as a set of entities by the constant Env using the equation: `eq Env = egf hrg e p t .`

Notably, all the experiments can be modeled as reachability properties that check for the presence of a specific molecule in an interactive process. For instance, the strategy `s(1,e t,p) ; se-steady(e t) ; includes(akt)` verifies the efficacy of the drug combination e t in the BT474 cancer subtype. This strategy executes an initial step using a context that must include e and t but excludes the drug p. This initial context may also contain any combination of the input stimuli egf and hrg required to activate the metabolic process. Subsequent steps are then executed using a context containing only e and t until a steady state (i.e., an attractor) is reached. Finally, the target molecule akt is checked within the attractor state. We can therefore run the strategic search from the initial state `< BT474 | empty | empty >`

```
srew < BT474 | empty | nil > using s(1,e t,p) ; se-steady(e t) ; in(akt) .
[...]
```

No solution.

```
rewrites: 12346 in 1ms cpu (1ms real) (10588336 rewrites/second)
```

that did not find any solution, thus demonstrating the effectiveness of treatment that combines e and t. On the contrary, the following strategic search for the

combined drugs **e** and **p**, produces four solutions. Each of these serves as a valid counterexample highlighting the limitations of this drug combination.

```
srew < BT474 | empty | nil > using s(1,e p,t) ; se-steady(e p) ; in(akt) .
[...]
```

Solution 4

```
result State: < ... | empty, mtor plcg erk12, akt p70s6k mtor plcg erk12
                pkca, akt p70s6k mtor plcg erk12 pkca
                | e p egf hrg, e p, e p >
```

No more solutions.

rewrites: 13946 in 2ms cpu (2ms real) (6900544 rewrites/second)

Using similar strategies, we successfully reproduced all the results presented in Table 1. The execution times on a MacBook Pro M4 with 32 GB of RAM were negligible, requiring only a few milliseconds per experiment.

Beyond reproducing the results in [11], our framework is expressive enough to test new hypotheses and evaluate more complex scenarios.

Context Sequence Synthesis. Our framework enables the automatic synthesis of context sequences that satisfy a given property. This capability is particularly valuable when searching for potential drug treatments that are successful against a specific key molecule. Consider, for instance, a bounded strategic search over the computation tree of the HCC1954 cancer subtype. The objective is to identify all potential treatments that, after exactly 6 sessions, reach an attractor state where **akt** is absent. For this search, we require that the contexts consistently include the input stimulus **egf** and the drug **e**, while strictly excluding the input stimulus **hrp** and the drug **p**.

```
srew < HCC1954 | empty | nil > using s(6,egf e,hrp p) ; isSteady ;
                                not(in(erk12)) .
[...]
```

Solution 63

```
rewrites: 762112 in 43ms cpu (43ms real) (17637806 rewrites/second)
result State: < ... | empty, p70s6k pkca, akt p70s6k pkca, akt p70s6k
                mtor pkca, akt p70s6k mtor pkca, akt p70s6k mtor
                pkca, akt p70s6k mtor pkca | e t egf, e t egf,
                e t egf, e t egf, e t egf, e t egf >
```

No more solutions.

rewrites: 762112 in 47ms cpu (48ms real) (16163220 rewrites/second)

The search yields 63 solutions, where each context sequence in the final state represents a treatment satisfying the specified property.

Furthermore, instead of limiting the depth of the search space, we can just limit the number of solutions to be delivered. This can be achieved, e.g., by the search command

```
srew [10] < HCC1954 | empty | nil > using s(1,egf e,hrp p) + ; isSteady ;
                                not(in(erk12)) .
```

that exactly returns the first 10 solutions that meet the previously specified stimulus and drug constraints, while removing the depth restriction on the computation tree (i.e., the 6-session treatment limit).

Perfect Recall Context Strategies. Thanks to the `se-cond` operator, we can generate conditional context sequences that depend on the entire result sequence computed so far. This allows us to implement perfect recall strategies that generate new contexts based on the presence or absence of specific entities computed in past steps. As an example, consider the BT474 cancer subtype under the persistent input stimulus `egf`. Suppose we administer the drugs `e` and `t` for the first 5 treatment sessions to inhibit the `akt` molecule; we want to determine whether it is possible to drop `t` from the treatment and still reach an attractor that does not contain `akt` if in the last 3 computed result sets `akt` is absent. We can specify this reachability problem by means of the following strategic search:

```
srew [1] < BT474 | empty | nil > using se(5, egf e t) ;
      se-cond(egf e, empty, akt, 3) + ; isSteady ; not (in(akt)) .
```

whose strategy first executes exactly five steps under the fixed context `egf e t`. Then, `se-cond(egf e, empty, akt, 3) +` repeatedly fires single steps with the context `egf e t`, but only as long as the three most recent result sets exclude `akt`. Finally, `isSteady` asserts that the system has reached an attractor, and `not(in(akt))` verifies that it does not contain `akt`. The search returns a solution confirming that we can safely withdraw drug `t` after five steps and still reach a stable configuration without `akt`.

LTL and CTL Model-checking. The `umaudemc` tool [15] is a general interface to various model checkers for Maude models. Here we demonstrate how it can be employed to formally verify LTL and CTL formulas against our models for HER2-positive cancers. Both LTL and CTL formulas are built using atomic formulas, Boolean connectives, and standard temporal operators. While LTL relies on the operators $\Box$ (always), $\Diamond$ (eventually), $\circ$ (next), $\mathbf{U}$ (until), and $\mathbf{R}$ (release) to reason about linear execution, CTL evaluates these properties over the branching paths of a computation tree by quantifying them using path quantifiers to state that a property holds either for **All** paths or that there **Exists** a path.

These temporal logic formulas can be model-checked via the `check` command of `umaudemc`, which executes the verification by taking as input the Maude specification file, the initial state defined as a term of sort `State`, the target temporal formula, and an optional Maude strategy parameter that allows properties to be model-checked exclusively on the fragment of the computation tree identified by that strategy, thereby reducing the search space.

More concretely, the results of [11], that we reproduced using the `srew` command, can alternatively be obtained via model-checking LTL formulas. For example, the command

```
umaudemc check SKBR3.mau " < SKBR3 | empty | nil > "
      "<> ( p70s6k /\ steady )" "s(1,e p,t) ; se-steady(e p)"
```

checks for the presence of `p70s6k` in attractors of the SKBR3 cancer subtype when drugs `e` and `p` are continuously administered. This is achieved by verifying the LTL formula $\Diamond(p70s6k \wedge \text{steady})$, where `p70s6k` and `steady` are atomic formulas respectively specifying the presence of `p70s6k` in a state and the encoding of the `isSteady` Boolean function as an atomic proposition. Note that the verification is performed only on a fragment of the computation tree for the initial state $\langle \text{SKBR3} \mid \text{empty} \mid \text{nil} \rangle$, identified by the strategy $s(1, e \ p, t) ; \text{se-steady}(e \ p)$. This strategy first performs a single step with a context that includes drugs `e` and `p` (but not `t`) plus any possible combination of input stimuli `egf` and `hrg`, then repeatedly applies the fixed context `e p` until the system reaches a steady state.

CTL formulas allow us to assess the efficacy of distinct treatments in parallel. Consider three 10-step treatments for the BT474 cancer subtype, administering drugs `e`, `p`, and `t` respectively under the persistent input stimulus `egf`. To ask whether at least one treatment admits a run reaching an attractor in which `akt` is absent, we verify the CTL formula $E \Box (\text{steady} \rightarrow \neg \text{akt})$ via the command

```
umaudemc check BT474.maude "< BT474 | empty | nil >"
"E [] ( steady -> ~ akt )" "se-sets(10,(egf e, egf p, egf t))"
```

The formula is satisfied, consistently with Table 1, which shows that `e` inhibits `akt` production in BT474. The universally quantified formula $A \Box (\text{steady} \rightarrow \neg \text{akt})$ would instead fail, since drugs `p` and `t` are not effective against `akt`.

6 Conclusions and Related Work

We have presented a formal, executable semantics for RSs implemented as a fully nondeterministic Maude rewrite theory. By leveraging the Maude strategy language, our framework successfully controls the combinatorial growth of unrestricted context generation, allowing computation trees to be pruned while isolating only relevant execution paths. The versatility of this approach has been demonstrated through various complex analysis tasks—including strategic reachability analysis, context sequence synthesis, perfect recall strategies, and LTL/CTL model checking. Our results have been validated against a real-size biological case study on drug treatments in HER2-positive breast cancer.

There are several analysis and verification frameworks for RSs that can be compared to ours. [1] provides a Maude formalization of RSs which is suitable for trace slicing, a technique used to simplify computation traces. The methodology in [1] needs context sequences to be manually given by the user, whereas our approach nondeterministically yields all possible context sequences, automatically.

Our previous work [2] describes the process algebra-based interaction language `ccReact`, which is well-suited for RS simulation and verification. Similar to our strategy-based approach, `ccReact` extends the classical RS model by supporting recursive, nondeterministic, and conditional context sequences. However, conditional context generation in `ccReact` depends strictly on the entities present in the current system state. In contrast, our new framework makes use of perfect

recall strategies to generate contexts based on the entire computation history. Furthermore, context sequence synthesis is not supported in `ccReact`.

[12] defines a CTL logic for context-restricted RSs, while [14] introduces an LTL logic for a multiset-based version of RSs with discrete concentrations. A key difference is that both [12, 14] rely on custom, ad-hoc model checkers. In contrast, our framework utilizes standard Maude model-checkers and avoids the need of developing/maintaining custom verification tools.

Is context synthesis supported in `ccReact`, Carlos?

References

1. Ballis, D., Brodo, L., Falaschi, M.: Modeling and Analyzing Reaction Systems in Maude. *Electronics* **13**(6,1139) (2024). <https://doi.org/https://doi.org/10.3390/electronics13061139>
2. Ballis, D., Brodo, L., Falaschi, M., Olarte, C.: `ccreact`: a rewriting framework for the formal analysis of reaction systems. *Int. J. Softw. Tools Technol. Transf.* **26**(6), 707–725 (2024). <https://doi.org/10.1007/S10009-024-00772-Z>, <https://doi.org/10.1007/s10009-024-00772-z>
3. Barbuti, R., Gori, R., Levi, F., Milazzo, P.: Investigating dynamic causalities in reaction systems. *Theor. Comput. Sci.* **623**, 114–145 (2016). <https://doi.org/https://doi.org/10.1016/j.tcs.2015.11.041>
4. Barbuti, R., Gori, R., Milazzo, P.: Encoding boolean networks into reaction systems for investigating causal dependencies in gene regulation. *Theoretical Computer Science* **881**, 3–24 (2021). <https://doi.org/https://doi.org/10.1016/j.tcs.2020.07.031>, special Issue on Reaction Systems
5. Brijder, R., Ehrenfeucht, A., Main, M., Rozenberg, G.: A tour of reaction systems. *Int. J. Found. Comput. Sci.* **22**(07), 1499–1517 (2011). <https://doi.org/https://doi.org/10.1142/S0129054111008842>
6. Clavel, M., Durán, F., Eker, S., Escobar, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.: Maude Manual (Version 3.5.1). Tech. rep. (2025), available at: <https://maude.lcc.uma.es/maude-manual/>
7. Corolli, L., Maj, C., Marinia, F., Besozzi, D., Mauri, G.: An excursion in reaction systems: From computer science to biology. *Theor. Comput. Sci.* **454**, 95–108 (2012). <https://doi.org/https://doi.org/10.1016/j.tcs.2012.04.003>
8. Durán, F., Eker, S., Escobar, S., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.L.: Programming and symbolic computation in maude. *J. Log. Algebraic Methods Program.* **110** (2020)
9. Ehrenfeucht, A., Main, M.G., Rozenberg, G.: Combinatorics of life and death for reaction systems. *Int. J. Found. Comput. Sci.* **21**(3), 345–356 (2010). <https://doi.org/10.1142/S0129054110007295>
10. Ehrenfeucht, A., Main, M.G., Rozenberg, G.: Functions defined by reaction systems. *Int. J. Found. Comput. Sci.* **22**(1), 167–178 (2011). <https://doi.org/10.1142/S0129054111007927>
11. der Heyde, S.V., Bender, C., Henjes, F., Sonntag, J., Korf, U., Beißbarth, T.: Boolean ErbB network reconstructions and perturbation simulations reveal individual drug response in different breast cancer cell lines. *BMC Systems Biology* **8**(1), 75 (2014). <https://doi.org/10.1186/1752-0509-8-75>
12. Męski, A., Penczek, W., Rozenberg, G.: Model checking temporal properties of reaction systems. *Information Sciences* **313**, 22–42 (2015). <https://doi.org/10.1016/j.ins.2015.03.048>

13. Meseguer, J.: Twenty years of rewriting logic. *J. Log. Algebraic Methods Program.* **81**(7-8), 721–781 (2012). <https://doi.org/10.1016/J.JLAP.2012.06.003>
14. Meski, A., Koutny, M., Penczek, W.: Verification of linear-time temporal properties for reaction systems with discrete concentrations. *Fundam. Informaticae* **154**(1-4), 289–306 (2017). <https://doi.org/10.3233/FI-2017-1567>
15. Rubio, R., Martí-Oliet, N., Pita, I., Verdejo, A.: Strategies, model checking and branching-time properties in maude. *J. Log. Algebraic Methods Program.* **123**, 100700 (2021). <https://doi.org/10.1016/J.JLAMP.2021.100700>
16. Rubio, R., Martí-Oliet, N., Pita, I., Verdejo, A.: Strategies, Model checking and Branching-time Properties in Maude. *J. Log. Algebraic Methods Program.* **123**, 100700 (2021)